

Design a News Feed System - Wrap Up

1. Overview

In this chapter, we designed a **News Feed System** similar to:

- Facebook Feed
- Instagram Feed
- Twitter/X Timeline

The system mainly consists of:

- **Feed Publishing**
- **News Feed Retrieval**

2. Main Components Discussed

Feed Publishing Flow

When a user creates a post:

1. Request goes through API servers
2. Post is stored in database/cache
3. Fanout service distributes post to friends
4. Notification service alerts users

News Feed Retrieval Flow

When a user opens the feed:

1. Request goes to news feed service
2. Feed IDs are fetched from cache
3. Post/user data are hydrated
4. Feed returned to client

3. Fanout Models Discussed

Fanout on Write (Push Model)

News feed is precomputed during write time.

Pros

- Fast feed retrieval
- Real-time updates

Cons

- Expensive for celebrity users
- Hotkey problem

Fanout on Read (Pull Model)

Feed generated during read time.

Pros

- Better for inactive users
- Avoids hotkey issue

Cons

- Slower feed generation

Hybrid Approach

System combines both approaches:

- Push model → normal users
- Pull model → celebrity/high follower users

4. Cache Architecture

Cache is critical for performance.

The cache layer includes:

Cache Layer	Purpose
News Feed Cache	Stores feed IDs
Content Cache	Stores post data
Social Graph Cache	Stores friendships
Action Cache	Likes, comments, replies
Counter Cache	Followers, likes count

5. Important Design Considerations

Scalability

Large-scale systems require:

- Horizontal scaling
- Stateless web servers
- Distributed caches
- Message queues

Database Scaling

Possible approaches:

- Vertical scaling
- Horizontal scaling (Sharding)
- Master-Slave replication
- Read replicas

Caching

Heavy caching is necessary because:

- News feed is read-heavy
- Reduces DB load
- Improves latency

Multi Data Center Support

System should support:

- Geo-distribution
- Fault tolerance
- Regional failover

Message Queues

Used to:

- Decouple components
- Handle asynchronous processing
- Improve reliability

Monitoring

Important metrics include:

- Peak QPS
- Feed refresh latency
- Cache hit rate
- Fanout delays

6. Key Takeaways

A News Feed System is mainly a balance between:

- Fast reads
- Efficient writes
- Scalability
- Low latency
- Storage optimization

Hybrid fanout strategy is commonly used in real-world systems.

Cache plays the most important role in performance optimization.

Final Conclusion

A scalable News Feed System requires efficient feed publishing, optimized feed retrieval, distributed caching, fanout management, and scalable storage architecture. By combining push and pull models, using caches extensively, and leveraging asynchronous processing with message queues, the system can support millions of users with low latency and high availability.